Verslag practicum AI:

Noodhulpsysteem

door: Nik Van Daele

https://github.com/nikvandaele/AI-opdracht-1

project setup:

Beginnen bij het begin, dat is te zeggen, bij het eerste prompt. Ik gaf de pdf met de opracht aan google gemini en vroeg het te helpen met het opstellen van een projectstructuur.

Zijn antwoord leek mij overkill dus ik besloot om zelf maar wat files aan te maken (run.py, main.py en NHS.py), of dit kiezen voor een minder ordelijke projectstructuur al dan niet een goede keuze was hangt waarschijnlijk af van of ik deze code ooit nog ergens anders voor nodig zal hebben, ik hoop alvast van niet.

A zoekalgorithme:*

Omdat ik sommige van de andere onderdelen van het project niet echt zag zitten besloot ik om te beginnen met het zoekalgorithme. Ik koos A* omdat ik dacht dat dat moest van de opdracht, blijkt dat dat niet zo was maar geen probleem want A* is natuurlijk het meest gebruikte zoekalgoritme en een van de beste opties.

Ik had niet echt zin om grote stukken van dit project door het LLM te laten genereren dus ik zocht wat rond online en vond de GeeksforGeeks-pagina over het algoritme[1] aan de hand van de pseudocode die daar te vinden is slaagde ik erin het algoritme te implementeren.

Ik koos voor de euclidische afstand als heuristiek aangezien inze straten in elke richting kunnen liggen en die daarom dus de beste aproximatie is, dit vond ik ook op de GeeksforGeeks pagina

Pas helemaal op het einde van deze implementatie komt het LLM terug voor in het verhaal. Ik had wat problemen met een infinite loop veroorzaakt door dat ik telkens ik g wou berekenen een vreemde aangepaste versie van mijn predecessor-graaf doorliep die regelmatig lussen bevatte.

Het LLM wees erop dat ik beter voor elke knoop de g waarde zou bijhouden zodat ik de g waardes van de opvolgers(successors) zo gemakkelijker kon berekenen. Het naar een sterrebeeld vernoemde LLM bewijst voor het eerst niet volledig nutteloos te zijn.

Bayes en HMM:

We zetten ons verhaal verder bij het derde promt. Ik vroeg het LLM of het suggesties had voor libraries om te gebruiken bij de implementatie van de bayesiaanse en hidden markov modellen.

Het antwoorde met suggesties maar gaf ook aan dat voor deze use case het niet zo ingewikkeld was om deze gewoon zelf te implementeren door gebruik te maken van numpy. Verzekerd door de

1 https://www.geeksforgeeks.org/dsa/a-search-algorithm/

woorden van het LLM met astrologisch geïnspireerde naam, besloot ik het te vragen om in grote lijnen te schetsen hoe ik dat zou doen voor het bayesiaans model en ga met die schets aan de slag.

Voor die wegen waarvoor er geen meldingen van het weer of de staat van de weg zijn besloot ik om aan de hand van de initiële weerkansen en de kansen gegeven het weer te bepalen wat de meest voorkomende wegconditie is en die conditie dan aan alle wegen zonder meldingen toe te schrijven.

Verder koos ik ervoor om de meldingen voor beide richtingin van een straat samen te nemen waardoor de conditie van een bepaalde straat altijd dezelfde is in beide richtingen, aangezien dat de richting waarin je door een straat rijd doorgaans geen invloed heeft op of die straat al dan niet beschadigd of verwoest is

Aangezien het voor bayes zo goed was gegaan besloot ik om ook voor HMM het LLM in grote lijnen te laten schetsen hoe ik dat zou doen, deze keer deed het het minder goed en genereerde het eigelijk gewoon de hele viterbi functie, ik heb hier best wel wat van overgenomen maar aangezien het LLM vernoemd naar lichtpuntjes in de nacht weinig tot geen context had over hoe ik mijn code graag schrijf of over de klasse waarin die functie zich bevind hab ook nog wel wat aanpassingen moeten maken.

Hier heb ik er ook weer voor gekozen om voor de wegen zonder meldingen toch een verkeersdrukte toe te wijzen gebaseerd op de initiële kansen die gegeven waren.

Voor diegenen die wel meldingen hebben koos ik ervoor om met de meest waarschijnlijke laatste verkeersdrukte te werken omdat dat de recentste info is die we hebben.

Verkeersdrukte is wel richtingsgebonden.

Voor de invloed van zowel wegconditie als verkeersdrukte op de reistijd koos ik ervoor om de telkens de snelheid op die wegen aan te passen dus voor een congested weg was het snelheid * 2/3, voor medium vrekeersdrukte snelheid * 0.9 en voor hoge verkeersdrukte snelheid * 2/3. Open conditie en lage verkeersdrukte hebben geen invloed op de snelheid en in het geval van een geblokeerde weg wordt die weg gewoon uit de graaf verwijderd.

Constraint based allocation:

Omdat het opzet van het allocatieprobleem mij zeer gelijkaardig leek aan processor scheduling uit onze cursus OS van vorig jaar ben ik eerst daar eens gaan kijken, maar na wat grasduinen door die cursus an een paar duckduckgo zoekopdrachten bleek dat ik daar voor dit specifiek geval geen kant en klare oplossing ging vinden en dus keerde ik me terug naar het LLM vernoemd naar een cluster lichtgevende kernfusiereactoren.

Uit de vraag een vergelijking op te maken tussen het gebruiken van een library en zelf iets in elkaar te steken. Opnieuw antwoorde het dat een library zoals ORtools mogelijks wat overkill was en een zelf geîmplementeerde oplossing met een greedy heuristiek een mogelijks goed alternatief was.

En dus besloot ik om het wederom zelf te doen, ik koos ervoor om eerst alle nog niet afgehandelde maar al wel beschikbare incidenten te sorteren op prioritieit om daarna voor elke prioriteitsklasse van hoog naar laag:

Voor elk incident te bepalen welke hulpverleners er niet druk bezig zijn met iets anders en de kwalificaties hebben om het af te handelen, alsook te bepalen welke routes deze zouden nemen en

hoe lang ze onderweg zouden zijn. Als het blijkt dat een hulpverlener er niet op tijd kan geraken wordt die als onbeschikbaar beschouwd.

Dan het incident in de prioriteitsklasse met de vroegste deadline te selecteren en de beschikbare gekwalificeerde hulpverlener die daar het snelst geraakt eropaf te sturen.

En dat tot we geen onbehandelde incidenten of beschikbare gekwalificeerde hulpverleners meer hebben.

Als ik even terugstap naar de cursus OS van vorig jaar zien we dat puur earliest deadline first optimaal is om de maximale delay te minimaliseren maar gemakkelijk veel processen een beetje te laat kan laten komen. Als wilt optimaliseren voor hoeveelheid processen die op tijd aankomen moet je moore-hodgeson toepassen waarbij je processen die te laat aankomen uit de EDF lijst verwijderd tot er geen meer te laat aankomen.

Gelijkaardig zorg ik er door geen hulpverleners die te laat zouden aankomen toe te laten voor dat er geen tijd verspilt wordt aan incidenten waar niemand op tijd kan geraken.

Dit is mogelijk een punt waar de realiteit en de simulatie een beetje uiteen lopen, in een reëel scenario zou je mischien liever hebben dat hulpverleners overal wat laat aankomen dan dat ze sommige oproepen volledig overslaan. Hoewel het sorteren op prioriteit er wel voor zorgt dat vooral de lagere prioriteiten worden overgeslagen.

Problemen en fouten:

Verder dan het feit dat het greedy allocatie algoritme mischien net iets te agressief is in het negeren van incidenten waar niemand strikt op tijd geraakt, denk ik ook dat de algemene projectstructuur beter had gekund om de code allemaal wat duidelijker te maken.

Ook had ik graag meer en betere testen geschreven maar door de tijdsdruk schieten die ook wel wat te kort vrees ik.

Persoonlijk nawoordje:

De vereiste om een LLM te gebruiken is in alle eerlijkheid het meest demotiverende ding dat ik ooit al in een opdracht heb zien staan.

De algemene problemen met mijn projectstructuur zijn hier deels aan te wijten omdat ik gewoon echt geen zin in had om aan het project te beginnen, laat staan om het deftig te doen.

Ik heb met een stevige portie koppigheid, uiteindelijk toch motivatie kunnen halen uit mijn frustraties met de opgave, maar ik hed dit toch graag anders gezien

volledige gresprek https://share.gemini.google/lVY3h8kh1UlK:

eerste prompt waarin ik het vraag van iets te doen:

“help ees met het opstellen van een projectstructuur hiervoor” met de opgave in bijlage

eerste prompt waarvan ik effectief dingen heb gebruikt:

mijn volledige A* implmentatie, gevolgd door:

“ik heb dit al als implementatie voor A* maar ik heb nog een probleem bij de ouders/ tempouders dat leidt tot een infinite loop in getPad“

verdere prompts:

“heb jij voorstellen voor mogelijke libraries die ik zou kunnen gebruiken voor de bayesian reasoning en de hidden markov models“

“kan je dan even in grote lijnen schetsen hoe ik met numpy bayesian reasoning zou doen”

“zou even hetzelfde kunnen doen voor het hidden markov model”

“kan je een vergelijking maken tussen libraries en zelf doen voor constraint based allocation”

“welke aloritmes zou je voorsdtellen om eht zelf te doen”

“het project is grootendeels af, kan jij er wat testen voor schrijven”

“kan je pytest gebruiken voor de tests”